

```

=====
Q01_Array_Max_Min_Average.cpp
=====
// Assignment 2 - Question 1
// Name: Zawar Shah
// Registration No: BF25NWELE0677

#include <iostream>
using namespace std;

int main() {
    const int SIZE = 10;
    int numbers[SIZE];
    int maximumValue, minimumValue, sum = 0;
    double average;

    cout << "Enter 10 integer values:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << "Value " << i + 1 << ": ";
        cin >> numbers[i];
    }

    maximumValue = numbers[0];
    minimumValue = numbers[0];

    for (int i = 0; i < SIZE; i++) {
        if (numbers[i] > maximumValue) {
            maximumValue = numbers[i];
        }

        if (numbers[i] < minimumValue) {
            minimumValue = numbers[i];
        }

        sum += numbers[i];
    }

    average = static_cast<double>(sum) / SIZE; // Convert to double to get decimal average

    cout << "\nMaximum value = " << maximumValue << endl;
    cout << "Minimum value = " << minimumValue << endl;
    cout << "Average = " << average << endl;

    return 0;
}

```

```

=====
Q02_Reverse_One_D_Array_Using_Function.cpp
=====
// Assignment 2 - Question 2
// Name: Zawar Shah
// Registration No: BF25NWELE0677

#include <iostream>
using namespace std;

void displayArray(const int array[], int size) {
    for (int i = 0; i < size; i++) {
        cout << array[i] << " ";
    }
}

```

```

        cout << endl;
    }

void reverseArray(int array[], int size) {
    for (int start = 0, end = size - 1; start < end; start++, end--) {
        int temp = array[start];
        array[start] = array[end];
        array[end] = temp;
    }
}

int main() {
    const int SIZE = 10;
    int numbers[SIZE];

    cout << "Enter 10 integers for the array:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << "Value " << i + 1 << ": ";
        cin >> numbers[i];
    }

    cout << "\nArray before reversal: ";
    displayArray(numbers, SIZE);

    reverseArray(numbers, SIZE); // Reverse the original array through a function

    cout << "Array after reversal: ";
    displayArray(numbers, SIZE);

    return 0;
}

```

```

=====
Q03_Matrix_Addition.cpp
=====

```

```

// Assignment 2 - Question 3
// Name: Zawar Shah
// Registration No: BF25NWELE0677

```

```

#include <iostream>
using namespace std;

```

```

int main() {
    const int ROWS = 3;
    const int COLS = 3;
    int matrixA[ROWS][COLS];
    int matrixB[ROWS][COLS];
    int result[ROWS][COLS];

    cout << "Enter elements of first 3x3 matrix:" << endl;
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            cout << "A[" << i << "][" << j << "]: ";
            cin >> matrixA[i][j];
        }
    }

    cout << "\nEnter elements of second 3x3 matrix:" << endl;
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {

```

```

        cout << "B[" << i << "][" << j << "]: ";
        cin >> matrixB[i][j];
    }
}

for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < COLS; j++) {
        result[i][j] = matrixA[i][j] + matrixB[i][j]; // Add matching elements
    }
}

cout << "\nResultant matrix after addition:" << endl;
for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < COLS; j++) {
        cout << result[i][j] << "\t";
    }
    cout << endl;
}

return 0;
}

```

```

=====
Q04_Sort_Student_Names.cpp
=====
// Assignment 2 - Question 4
// Name: Zawar Shah
// Registration No: BF25NWELE0677

#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE = 5;
    string names[SIZE];

    cout << "Enter names of 5 students:" << endl;
    cin.ignore();
    for (int i = 0; i < SIZE; i++) {
        cout << "Student " << i + 1 << ": ";
        getline(cin, names[i]);
    }

    for (int i = 0; i < SIZE - 1; i++) {
        for (int j = 0; j < SIZE - i - 1; j++) {
            if (names[j] > names[j + 1]) {
                string temp = names[j];
                names[j] = names[j + 1];
                names[j + 1] = temp;
            }
        }
    }

    cout << "\nNames in alphabetical order:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << names[i] << endl;
    }

    return 0;
}

```

```
}
```

```
=====
Q05_Power_System_Load_Analysis.cpp
=====
```

```
// Assignment 2 - Question 5
```

```
// Name: Zawar Shah
```

```
// Registration No: BF25NWELE0677
```

```
#include <iostream>
```

```
using namespace std;
```

```
double calculateTotalLoad(const double loads[], int size) {
    double total = 0;
    for (int i = 0; i < size; i++) {
        total += loads[i];
    }
    return total;
}
```

```
double findPeakLoad(const double loads[], int size) {
    double peak = loads[0];
    for (int i = 1; i < size; i++) {
        if (loads[i] > peak) {
            peak = loads[i];
        }
    }
    return peak;
}
```

```
int main() {
    const int HOURS = 24;
    double loadData[HOURS];

    cout << "Enter load demand in MW for 24 hours:" << endl;
    for (int i = 0; i < HOURS; i++) {
        cout << "Hour " << i + 1 << ": ";
        cin >> loadData[i];
    }

    double totalLoad = calculateTotalLoad(loadData, HOURS);
    double peakLoad = findPeakLoad(loadData, HOURS);

    cout << "\nTotal load for the day = " << totalLoad << " MW" << endl;
    cout << "Peak load = " << peakLoad << " MW" << endl;

    return 0;
}
```

```
=====
Q06_Student_Marks_Analysis.cpp
=====
```

```
// Assignment 2 - Question 6
```

```
// Name: Zawar Shah
```

```
// Registration No: BF25NWELE0677
```

```
#include <iostream>
```

```
#include <string>
```

```

using namespace std;

int findTopperIndex(const int marks[], int size) {
    int topperIndex = 0;
    for (int i = 1; i < size; i++) {
        if (marks[i] > marks[topperIndex]) {
            topperIndex = i;
        }
    }
    return topperIndex;
}

void displayPassFailStatus(const string names[], const int marks[], int size) {
    cout << "\nPass/Fail Status:" << endl;
    for (int i = 0; i < size; i++) {
        cout << names[i] << " = " << (marks[i] >= 50 ? "Pass" : "Fail") << endl;
    }
}

int searchStudentByName(const string names[], int size, const string& targetName) {
    for (int i = 0; i < size; i++) {
        if (names[i] == targetName) {
            return i;
        }
    }
    return -1;
}

int main() {
    const int SIZE = 5;
    string names[SIZE];
    int marks[SIZE];
    string searchName;

    cout << "Enter names and marks of 5 students:" << endl;
    cin.ignore();
    for (int i = 0; i < SIZE; i++) {
        cout << "\nStudent " << i + 1 << " name: ";
        getline(cin, names[i]);

        cout << "Marks of " << names[i] << ": ";
        cin >> marks[i];
        cin.ignore();
    }

    int topperIndex = findTopperIndex(marks, SIZE);

    cout << "\nTopper = " << names[topperIndex]
        << " with marks " << marks[topperIndex] << endl;

    displayPassFailStatus(names, marks, SIZE);

    cout << "\nEnter student name to search: ";
    getline(cin, searchName);

    int foundIndex = searchStudentByName(names, SIZE, searchName);
    if (foundIndex != -1) {
        cout << searchName << " found with marks " << marks[foundIndex] << endl;
    } else {
        cout << searchName << " not found in the record." << endl;
    }
}

```

```

    return 0;
}

```

```

=====
Q07_Renewable_Energy_Monitoring_System.cpp
=====

```

```

// Assignment 2 - Question 7
// Name: Zawar Shah
// Registration No: BF25NWELE0677

```

```

#include <iostream>
using namespace std;

```

```

double calculateTotalEnergy(const double energy[], int size) {
    double total = 0;
    for (int i = 0; i < size; i++) {
        total += energy[i];
    }
    return total;
}

```

```

int findMaximumGenerationHour(const double energy[], int size) {
    int maxIndex = 0;
    for (int i = 1; i < size; i++) {
        if (energy[i] > energy[maxIndex]) {
            maxIndex = i;
        }
    }
    return maxIndex;
}

```

```

double calculateAverageEnergy(const double energy[], int size) {
    return calculateTotalEnergy(energy, size) / size;
}

```

```

int main() {
    const int HOURS = 24;
    double energyGenerated[HOURS];

    cout << "Enter energy generation data in kWh for 24 hours:" << endl;
    for (int i = 0; i < HOURS; i++) {
        cout << "Hour " << i + 1 << ": ";
        cin >> energyGenerated[i];
    }

    double totalEnergy = calculateTotalEnergy(energyGenerated, HOURS);
    int maxHourIndex = findMaximumGenerationHour(energyGenerated, HOURS);
    double averageEnergy = calculateAverageEnergy(energyGenerated, HOURS);

    cout << "\nTotal energy generated = " << totalEnergy << " kWh" << endl;
    cout << "Hour with maximum generation = " << maxHourIndex + 1 << endl;
    cout << "Average energy generation = " << averageEnergy << " kWh" << endl;

    return 0;
}

```

```

=====
Q08_Battery_Health_Monitoring.cpp
=====

```

=====

```
// Assignment 2 - Question 8
// Name: Zawar Shah
// Registration No: BF25NWELE0677
```

```
#include <iostream>
using namespace std;
```

```
int countUnsafeReadings(const double voltages[], int size, double minimumSafe, double maximumSafe) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (voltages[i] < minimumSafe || voltages[i] > maximumSafe) {
            count++;
        }
    }
    return count;
}
```

```
double findMinimumVoltage(const double voltages[], int size) {
    double minimumValue = voltages[0];
    for (int i = 1; i < size; i++) {
        if (voltages[i] < minimumValue) {
            minimumValue = voltages[i];
        }
    }
    return minimumValue;
}
```

```
double findMaximumVoltage(const double voltages[], int size) {
    double maximumValue = voltages[0];
    for (int i = 1; i < size; i++) {
        if (voltages[i] > maximumValue) {
            maximumValue = voltages[i];
        }
    }
    return maximumValue;
}
```

```
int main() {
    const int SIZE = 20;
    const double MIN_SAFE_VOLTAGE = 3.0;
    const double MAX_SAFE_VOLTAGE = 4.2;
    double voltages[SIZE];

    cout << "Enter 20 voltage readings of the battery:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << "Reading " << i + 1 << ": ";
        cin >> voltages[i];
    }

    int unsafeReadings = countUnsafeReadings(voltages, SIZE, MIN_SAFE_VOLTAGE, MAX_SAFE_VOLTAGE);
    double minimumVoltage = findMinimumVoltage(voltages, SIZE);
    double maximumVoltage = findMaximumVoltage(voltages, SIZE);

    cout << "\nMinimum voltage = " << minimumVoltage << " V" << endl;
    cout << "Maximum voltage = " << maximumVoltage << " V" << endl;
    cout << "Unsafe readings count = " << unsafeReadings << endl;

    if (unsafeReadings > 0) {
        cout << "Warning: Unsafe voltage readings exist." << endl;
    } else {
        cout << "All voltage readings are within the safe range." << endl;
    }
}
```

```

    }

    return 0;
}

```

```

=====
Q09_Smart_Grid_Load_Distribution.cpp
=====

```

```

// Assignment 2 - Question 9
// Name: Zawar Shah
// Registration No: BF25NWELE0677

```

```

#include <iostream>
using namespace std;

```

```

void calculateRegionTotals(const double consumption[][7], int rows, int cols, double totals[])
{
    for (int i = 0; i < rows; i++) {
        totals[i] = 0;
        for (int j = 0; j < cols; j++) {
            totals[i] += consumption[i][j];
        }
    }
}

```

```

int findHighestDemandDay(const double consumption[][7], int rows, int cols) {
    int highestDayIndex = 0;
    double highestTotal = 0;

    for (int day = 0; day < cols; day++) {
        double dayTotal = 0;
        for (int region = 0; region < rows; region++) {
            dayTotal += consumption[region][day];
        }

        if (day == 0 || dayTotal > highestTotal) {
            highestTotal = dayTotal;
            highestDayIndex = day;
        }
    }

    return highestDayIndex;
}

```

```

double calculateOverallAverage(const double consumption[][7], int rows, int cols) {
    double total = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            total += consumption[i][j];
        }
    }
    return total / (rows * cols);
}

```

```

int main() {
    const int REGIONS = 4;
    const int DAYS = 7;
    double consumption[REGIONS][DAYS];
    double regionTotals[REGIONS];

    cout << "Enter power consumption in MW for 4 regions over 7 days:" << endl;
}

```



```

    for (int i = 0; i < REGIONS; i++) {
        for (int j = 0; j < DAYS; j++) {
            cout << "Region " << i + 1 << ", Day " << j + 1 << ": ";
            cin >> consumption[i][j];
        }
    }

    calculateRegionTotals(consumption, REGIONS, DAYS, regionTotals);
    int highestDemandDay = findHighestDemandDay(consumption, REGIONS, DAYS);
    double overallAverage = calculateOverallAverage(consumption, REGIONS, DAYS);

    cout << "\nTotal consumption for each region:" << endl;
    for (int i = 0; i < REGIONS; i++) {
        cout << "Region " << i + 1 << " = " << regionTotals[i] << " MW" << endl;
    }

    cout << "\nDay with highest total demand = Day " << highestDemandDay + 1 << endl;
    cout << "Overall average consumption = " << overallAverage << " MW" << endl;

    return 0;
}

```

```

=====
Q10_Classroom_Attendance_System.cpp
=====

```

```

// Assignment 2 - Question 10
// Name: Zawar Shah
// Registration No: BF25NWELE0677

```

```

#include <iostream>
using namespace std;

```

```

void calculateStudentAttendance(const int attendance[][7], int rows, int cols, int totals[])
{
    for (int i = 0; i < rows; i++) {
        totals[i] = 0;
        for (int j = 0; j < cols; j++) {
            totals[i] += attendance[i][j];
        }
    }
}

```

```

int findHighestAttendanceStudent(const int totals[], int size) {
    int highestIndex = 0;
    for (int i = 1; i < size; i++) {
        if (totals[i] > totals[highestIndex]) {
            highestIndex = i;
        }
    }
    return highestIndex;
}

```

```

double calculateAttendancePercentage(const int attendance[][7], int rows, int cols) {
    int presentCount = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            presentCount += attendance[i][j];
        }
    }

    return (static_cast<double>(presentCount) / (rows * cols)) * 100;
}

```

```

}

int main() {
    const int STUDENTS = 5;
    const int DAYS = 7;
    int attendance[STUDENTS][DAYS];
    int studentTotals[STUDENTS];

    cout << "Enter attendance for 5 students over 7 days (1 for present, 0 for absent):" << endl;
    for (int i = 0; i < STUDENTS; i++) {
        for (int j = 0; j < DAYS; j++) {
            cout << "Student " << i + 1 << ", Day " << j + 1 << ": ";
            cin >> attendance[i][j];
        }
    }

    calculateStudentAttendance(attendance, STUDENTS, DAYS, studentTotals);
    int highestAttendanceStudent = findHighestAttendanceStudent(studentTotals, STUDENTS);
    double attendancePercentage = calculateAttendancePercentage(attendance, STUDENTS, DAYS);

    cout << "\nTotal attendance of each student:" << endl;
    for (int i = 0; i < STUDENTS; i++) {
        cout << "Student " << i + 1 << " = " << studentTotals[i] << " days" << endl;
    }

    cout << "\nStudent with highest attendance = Student " << highestAttendanceStudent + 1 << endl;
    cout << "Overall class attendance percentage = " << attendancePercentage << "%" << endl;

    return 0;
}

```